

NIFTY Options Signal Pod

AI Research Engineer Intern Screening Report

Candidate: Kavyansh Gupta **Track:** AI-SLM **Base model:** TinyLlama-1.1B-Chat-v1.0

Repository: github.com/kavyansh34/nifty-options-signal-pod

Kaggle notebook: kaggle.com/kavyansh34/nifty-signal-pod-v2

Executive Summary

I built a small language model signal pod for NIFTY options snapshots and wrapped it in a deterministic orchestrator. The orchestrator applies ADX suppression, strict JSON/schema validation, and a low-conviction downgrade before anything reaches downstream systems.

Final position: this is a useful research prototype, not a production-ready trading pod. The deterministic layer behaves in the spirit of the brief, but the model did not earn permission to trade: accuracy is below the committed threshold, the RAG parse-failure rate still misses the strict threshold in one window, conviction is not calibrated, and the VIX-regime gap cannot be measured from the submitted aggregate artifacts. I would rather report that honestly than make the report look stronger than the evidence.

The work should be read as an honest screening artifact: it shows how I inspected the supplied data, made the schema safe, tracked experiments, and then used the evaluation results to decide against deployment. The most important outcome is not a high accuracy number; it is the conclusion that the current pod should remain behind deterministic controls until row-level evidence proves otherwise.

Pre-Committed Evaluation Matrix

Committed item	Before-run threshold	Observed evidence	Verdict
Directional accuracy	>52% on non-NEUTRAL CE/PE calls	Baseline windows: 23.9%-42.9%; RAG windows: 17.6%-45.5%	FAIL
Schema / parse reliability	Schema pass >95%; parse failure <2%	RAG parse failures: 0%-3.1%; one window remains above the 2% pass threshold	FAIL
Conviction validity	Higher conviction bins should have higher realized accuracy	Predictions concentrate in 0.50-0.60, leaving too few bins to calibrate	NO DATA
VIX regime gap	High-VIX vs low-VIX accuracy gap <8 pp	Row-level VIX buckets were not logged, so the gap cannot be measured honestly	NO DATA
Orchestrator logging	Reason code on every decision	Implemented for ADX suppression, parse/schema failure, low conviction, and pass-through	PASS
ADX suppression rule	ADX <20 returns NEUTRAL without model call	Suppression fired in low-trend windows: 80% on days 41-45 and 40% on days 46-50	PASS

The matrix is deliberately strict. A trading pod can pass formatting checks and still fail as a research system if confidence does not mean anything, or if regime performance cannot be measured. I therefore treat the two NO DATA rows as unresolved risk, not as neutral omissions.

1. Eval Suite Design

The eval suite was designed before training. I used walk-forward evaluation only: days 31-60 split into six 5-day blocks. I avoided k-fold because using future days to evaluate earlier days would create look-ahead bias. Accuracy is measured only on CE/PE calls because NEUTRAL is a suppression decision, not a directional forecast.

Metric	Pass	Warn	Fail / Unsafe
Directional accuracy	>52%	48%-52%	<48%
Schema pass rate	>95%	90%-95%	<90%
Parse failure rate	<2%	2%-5%	>5%
Conviction validity	Monotonic accuracy by conviction bin	Mostly monotonic	Flat, inverted, or too sparse
VIX regime gap	<8 percentage points	8-15 pp	>15 pp or not measurable

The important design choice was to treat conviction as a field that must earn meaning. The model writes a number as text. A next-token softmax can tell me whether the token "0.6" was likely after the prompt; it cannot tell me whether a CE or PE call is reliable. I therefore planned to judge conviction empirically: as conviction rises, realized directional accuracy should rise too, and that relationship should not disappear in high-VIX slices.

The orchestrator metrics were part of the eval suite from the start because downstream systems never consume raw pod output. A model that produces an attractive raw signal but is suppressed 80% of the time is not equivalent to a model that passes through cleanly. For that reason, I report suppression and downgrade rates alongside accuracy rather than treating them as implementation details.

2. Data Audit

The training file contained issues that would have directly broken the pod if I trained on it blindly. The audit mattered more than squeezing another training run out of Kaggle.

Finding	Where / size	Decision	Reasoning
Non-numeric conviction labels	Rows 47-91, 45 records: high, moderate, weak, strong, 0.8 (high)	Dropped these rows	Mapping words to floats would invent labels for the exact field the brief asked me to make meaningful.
Wrong schema key	All 300 records used generated_at instead of timestamp	Renamed to timestamp before training	Without this, the model would learn to violate the fixed output schema.
No ADX <20 examples	Zero cleaned records covered low-ADX conditions	Documented as unresolved	The orchestrator protects this region today, but the model itself has no learned behavior there.
Class imbalance	Clean set: NEUTRAL 113, CE 74, PE 68	Weighted CE/PE at 2x	The first behavior leaned toward safe/default output; weights helped without duplicate oversampling.

The key judgment call was the conviction block. I chose to drop bad rows rather than normalize them because the labels were not ordinal in a documented way. For example, mapping strong to 0.80 and moderate to 0.55 might look reasonable, but it would make the calibration experiment validate my own invented mapping rather than the data provider intent.

3. Fine-Tuning and RAG

I chose TinyLlama-1.1B over Phi-2 because the task is short structured generation, not broad reasoning. TinyLlama fits Kaggle T4 comfortably with QLoRA, leaves room for iteration, and is more plausible for CPU inference. I used LoRA rank 8, alpha 32, dropout 0.05, q/k/v/o target modules, 8 epochs, LR 5e-4, effective batch size 16, and 4-bit NF4 quantization.

Rank 8 was the compromise I wanted for this data size: rank 4 felt too restrictive for learning direction, horizon, and conviction together, while rank 16 would add capacity without enough clean examples to justify it. The final run strengthened the signal with alpha 32 and weighted CE/PE sampling, but I did not treat the lower training loss as proof of a better trading model; the walk-forward blocks remain the authority.

Input feature	Example value
nifty_spot / atm_iv / iv_skew_25d	21971.55 / 12.37 / 0.047
pcr / adx_14 / realized_vol_5d	1.01 / 28.1 / 9.81
vix_india / dte_nearest / moneyness_band	14.34 / 2 / ATM

Prompt and expected output

```
<|system|> You are a NIFTY options signal generator. Return valid JSON only with direction, conviction, horizon, signal_id, and timestamp.</s>
<|user|> Market snapshot: nifty_spot=21971.55, atm_iv=12.37, iv_skew_25d=0.047, pcr=1.01, adx_14=28.1, realized_vol_5d=9.81, vix_india=14.34, dte_nearest=2, moneyness_band=ATM. Signal:</s>
<|assistant|>
{"direction":"PE", "conviction":0.47, "horizon":"intraday", "signal_id":"...", "timestamp":"2024-10-01T09:15:00+05:30"}
```

Run	Configuration	Train loss	Lesson
v1	r=8, alpha=16, LR 2e-4, 3 epochs, unweighted	~1.35	Too weak; behavior leaned toward default/safe outputs.
v2 final	r=8, alpha=32, LR 5e-4, 8 epochs, CE/PE 2x	0.595	Stronger learning, but eval still did not clear the trust bar.

RAG was tested with `retrieve(market_state, k=3)`. It reduced parse failures compared with baseline and improved accuracy in some windows, but it did not make the pod safe. The retrieved context changed generation behavior; it did not create a stable relationship between context, direction, and conviction. My conclusion is that RAG needs a fixed token budget and row-level logging before it can be treated as a safe improvement.

The RAG prompt is useful only if the retrieved episodes remain subordinate to the current market state. In the next version I would cap each retrieved summary, include only outcome-relevant fields, and log whether the retrieved examples agreed or disagreed with the model direction. That would let me tell whether RAG is adding context or merely nudging the model toward a more verbose but still unreliable completion.

4. Results Against the Eval Suite

These results come from the committed MLflow JSON artifacts in the ZIP. The table keeps the RAG parse-failure rate because the RAG run is the condition I would compare against the final prompt experiment. Baseline parse failures were useful during debugging, but they are removed here to keep the report focused on the evaluated RAG condition.

Window	Baseline acc	RAG acc	RAG parse fail	ADX suppr.	RAG downgrade
31-35	0.239 CI 0.139-0.379	0.339 CI 0.233-0.463	1.5%	0.0%	0.0%
36-40	0.366 CI 0.236-0.519	0.373 CI 0.261-0.500	3.1%	0.0%	6.2%
41-45	0.429 CI 0.158-0.750	0.231 CI 0.082-0.503	0.0%	80.0%	0.0%
46-50	0.324 CI 0.191-0.492	0.300 CI 0.108-0.603	0.0%	40.0%	9.2%
51-55	0.255 CI 0.155-0.389	0.455 CI 0.213-0.720	0.0%	0.0%	24.6%
56-60	0.241 CI 0.146-0.369	0.176 CI 0.062-0.410	0.0%	0.0%	47.7%

Why conviction calibration failed: the model did not produce a useful spread of conviction values. Baseline valid directional calls sit almost entirely in 0.50-0.60; RAG creates a small 0.60-0.70 pocket, but not enough density across bins to estimate a reliability curve. The right conclusion is not "calibration looks okay"; it is that this conviction field is not yet informative enough to use as a risk control.

Why the VIX gap failed: the ZIP has aggregate window metrics but not per-row outputs joined to vix_india. A real high-VIX/low-VIX comparison needs each evaluated row to log timestamp, VIX, bucket, raw output, final output, reason code, and correctness. Without that, any exact VIX regime gap would be made up. The next run should fix the artifact design before changing the model.

The confidence intervals also matter. Several windows have small directional sample sizes after suppression and downgrades, so the intervals are wide. That tells me not to over-interpret any single window improvement from RAG. The safer reading is that RAG changed behavior, but the evidence is not yet strong enough to promote it to a default production prompt.

Result: the orchestrator is doing useful safety work, especially in low-ADX windows, but the pod fails the committed trust criteria.

5. Safety Walkthrough

Brief scenario: 09:30 on expiry Thursday, India VIX opens 3 standard deviations above its trailing 30-day mean, and ADX reads 14.

Stage 1, ingestion: the market-state dictionary carries `adx_14=14` and the elevated `vix_india` value into the orchestrator. Stage 2, regime check: ADX is checked before the model is called. Because $14 < 20$, the orchestrator returns NEUTRAL with conviction 0.0 immediately. Stage 3, logging: the reason is `ADX_BELOW_THRESHOLD`, trigger values include `{adx_14: 14.0, threshold: 20.0}`, `pod_output` is null, and `final_output` is the NEUTRAL fallback. Stage 4, downstream: the downstream pipeline only sees the orchestrator output, not any raw model text.

This exact path is safe, but the boundary is not. The high-VIX event is outside the training distribution, and expiry Thursday has microstructure effects that are not represented in the feature set: gamma pressure, fast IV decay, and dealer hedging behavior. If ADX printed 20.5 instead of 14, the model would be called in a regime it has not learned. The current orchestrator has no VIX circuit breaker, no expiry flag, and no out-of-distribution check. That is the gap I would close first.

This is the distinction the brief is testing: the ADX rule prevents one obvious failure, but it does not prove the pod understands the market. A safe system needs overlapping evidence. The deterministic rule should suppress known bad regimes, the model should have training coverage for nearby states, and the evaluation suite should show that both high-volatility and normal-volatility behavior are understood. My current system only satisfies the first of those three.

Next addition	Why it matters
VIX circuit breaker	Suppress when VIX is above a trailing statistical threshold, independent of model confidence.
Expiry-day guard	Raise conviction threshold or force NEUTRAL during opening expiry volatility.
Row-level eval logging	Make conviction calibration and VIX regime gaps measurable instead of inferred.
High-VIX / low-ADX examples	Teach the model that some regimes should produce NEUTRAL, not just rely on the wrapper.
RAG token budget	Keep retrieved context useful without destabilizing JSON generation.

The current system is safest when the deterministic layer says no. It is not yet safe when the model is allowed to speak in rare regimes.

Implementation Hurdles and Moves Taken

The practical route to this point was not smooth. These are the main hurdles and the moves I made:

Hurdle	Move taken
Kaggle dependency conflicts across torch, transformers, peft, and bitsandbytes	Stopped pinning packages already present in Kaggle; installed only missing trl/mlflow and recorded final versions after the run.
SFTTrainer API changes	Removed deprecated tokenizer, dataset_text_field, and max_seq_length arguments; used the installed API instead of older examples.
fp16/bf16 and TrainingArguments errors	Switched to bf16 and updated argument names for the installed transformers version.
CUDA illegal memory access after failed runs	Restarted the full Kaggle session, not only the notebook kernel, to clear GPU state.
dtype mismatch on adapter reload	Matched inference bfloat16 to training bfloat16.
Training typo: train_dataset() instead of train()	Caught by comparing unchanged outputs after supposed training; corrected to trainer.train().
Conviction and VIX evaluation gaps	Reported them as unresolved because short-time fixes should improve logging, not invent numbers.

Final Position

The project reached a working orchestrated prototype with adapter weights, MLflow artifacts, a data audit, and a clear failure analysis. It did not reach a trustworthy trading pod. The strongest next step is not another headline training run; it is better evidence: row-level evaluation, deterministic VIX/expiry guards, and a conviction design that produces a measurable distribution.

If I had one more short iteration, I would not spend it changing the base model. I would first fix the artifact gap: save per-row predictions, VIX bucket, ADX value, reason code, parse status, final direction, and correctness. That would immediately answer the two questions this report cannot answer cleanly today: whether conviction is meaningful and whether high-VIX behavior is materially worse than low-VIX behavior.